State-Space Models

2026-04-17

Introduction

State-space models describe a time series via a latent state vector that evolves over time, with observations as a noisy function of the state. This general framework encompasses essentially every classical time-series model — ARIMA, exponential smoothing (ETS), structural time series, hidden Markov models, dynamic linear models — as special cases. The unified framework provides a consistent inference machinery (the Kalman filter for linear Gaussian models; particle and extended-Kalman filters for non-linear or non-Gaussian extensions) and a flexible specification language for custom models.

Prerequisites

A working understanding of linear algebra, Bayesian or likelihood inference, and basic familiarity with classical ARIMA / ETS forecasting.

Theory

The standard linear Gaussian state-space model has two equations:

Observation equation: $y_t = F_t x_t + v_t, \quad v_t \sim \mathcal{N}(0, V_t)$.

State transition: $x_t = G_t x_{t-1} + w_t, \quad w_t \sim \mathcal{N}(0, W_t)$.

The matrices (F_t, G_t, V_t, W_t) specify the model — they are typically constant in time but can vary deterministically. Inference proceeds in three modes: **filtering** estimates $x_t \mid y_{1:t}$ in real time; **smoothing** estimates $x_t \mid y_{1:T}$ using the full series; **prediction** extends the state forward beyond the data.

The Kalman filter computes both filtered and smoothed states by recursive Gaussian updates; non-linear or non-Gaussian extensions use the extended or unscented Kalman filter, the particle filter, or fully Bayesian MCMC.

Assumptions

For the standard Kalman filter: linear Gaussian observation and state equations. Non-linearity or non-Gaussianity requires extended methods.

R Implementation

```r
library(dlm)

# Local level model (random walk + noise)
build_dlm <- function(theta)
  dlmModPoly(order = 1, dV = exp(theta[1]), dW = exp(theta[2]))

set.seed(2026)
y <- Nile
fit <- dlmMLE(y, parm = c(0, 0), build = build_dlm)
dlm_mod <- build_dlm(fit$par)
```

```r
# Filtering
filt <- dlmFilter(y, dlm_mod)
smo  <- dlmSmooth(filt)

plot(y)
lines(dropFirst(smo$s), col = "red", lwd = 2)
```

Output & Results

`dlmMLE()` estimates the variance components by maximum likelihood. `dlmFilter()` and `dlmSmooth()` produce filtered and smoothed state estimates. The smoothed latent series captures the gradual level evolution; for the Nile, it shows the famous step-down around 1900 corresponding to the Aswan Low Dam construction.

Interpretation

A reporting sentence: "A local-level state-space model of Nile river flow fit by maximum likelihood estimated observation variance 15{,}100 and state-transition variance 1{,}470 (signal-to-noise ratio 0.10), favouring gradual state evolution; the smoothed latent series captured a clear level shift in the late 1890s." Always state the model structure (local level, local linear trend, etc.) and the estimated variance components.

Practical Tips

- ARIMA and ETS have exact state-space representations; `forecast::Arima()` and `ets()` use them internally.
- Unknown latent states are estimated automatically by the Kalman filter; the user typically only specifies the model structure and estimates variance components by MLE.
- For non-linear or non-Gaussian models, use extended Kalman, unscented Kalman, or particle filters; `bayesforecast` and `bsts` provide Bayesian alternatives.
- Parameter estimation by MLE (`dlmMLE`) or Bayesian MCMC (`bsts`); for Bayesian setups, weakly informative priors on variance components stabilise inference when data are weak.
- State-space is the general framework; choose a specific model (local level, local linear trend, structural with seasonal, dynamic regression) based on substantive domain knowledge.
- For diagnostics, examine the standardised innovations (filter residuals); they should be approximately white noise under correct model specification.

R Packages Used

`dlm` for the canonical dynamic linear model framework; `KFAS` for fast Kalman filtering / smoothing including non-Gaussian extensions; `bsts` for Bayesian structural time series; `MARSS` for multivariate ARMA state-space models; `fable::SSM()` for tidyverts state-space modelling.

For Reviewers

What to look for in a paper using this method.

- **Common misapplications.**
- **Diagnostics that should be reported but often aren't.**
- **Red flags in tables and figures.**
- **What to verify.**
- **What an adequate Methods paragraph must contain.**

See also — labs in this chapter

- Time series basics